Universidad de Alicante
Escuela Politécnica Superior

TECNOLOGÍA Y ARQUITECTURA ROBÓTICA (TAR)

Práctica 2

Turtlebot 3 en Entorno

Autor: Hernández Delgado, Jaime
Álvarez Bernabeu, Daniel
Mazón Muñoz, Manuel Jesús

Curso: 2025–2026

Repositorio GitHub:
https://github.com/nickhernd/TAR

22 de marzo de 2026

Índice general

1. Parte 2: Resolución de un Laberinto **2**
1.1. Preguntas teóricas previas 2
1.2. Ejercicio 1: Resolución del Laberinto 1 (Waffle) 3
1.3. Ejercicio 2: Laberinto 2 y análisis de limitaciones 4
1.4. Ejercicio 3: Repetición con Turtlebot 3 Burger 5
1.5. Ejercicio 4: Laberinto propio 6

1. Parte 2: Resolución de un Laberinto

En esta parte se utiliza el Turtlebot 3 equipado con un sensor LiDAR para resolver un laberinto de manera autónoma. Se emplea el paquete `maze_pkg`, que contiene el nodo `res_maze.py` basado en un algoritmo de seguimiento de pared izquierda (*left-wall following*).

1.1 Preguntas teóricas previas

Pregunta 12: Topic y tipo de mensaje del LiDAR

El topic asociado al LiDAR del Turtlebot 3 es `/scan`.
El tipo de mensaje es `sensor_msgs/msg/LaserScan`, que contiene los campos principales:

- `angle_min`, `angle_max`, `angle_increment`: rango y resolución angular.
- `range_min`, `range_max`: límites de distancia medibles.
- `ranges[]`: array con las distancias medidas en cada ángulo.

Para verificarlo se puede ejecutar:

```
ros2 topic list
ros2 topic info /scan
ros2 topic echo /scan --once
```

Pregunta 13: Rangos de distancia, ángulo de escaneo y origen de referencia

Los valores del LiDAR del Turtlebot 3 Waffle y Burger son:

Parámetro	**Waffle**	**Burger**
Rango mínimo	0.12 m	0.12 m
Rango máximo	3.5 m	3.5 m
Ángulo de escaneo	360°	360°
Resolución angular	1°	1°

Cuadro 1.1: Características del LiDAR del Turtlebot 3

El origen de referencia del LiDAR es el propio sensor, montado en la parte superior del robot. El índice 0 del array `ranges[]` corresponde a la dirección de avance del robot (frente), y los índices aumentan en sentido antihorario: índice 90 ≈ izquierda, 180 ≈ atrás, 270 ≈ derecha.

[CAPTURA: salida de `ros2 topic echo /scan -once` mostrando range_min, range_max, angle_min, angle_max]

Figura 1.1: Información del topic `/scan` del Turtlebot 3

1.2 Ejercicio 1: Resolución del Laberinto 1 (Waffle)

Se ha desarrollado el nodo `res_maze.py` dentro del paquete `maze_pkg`. El algoritmo implementado es *left-wall following* con una máquina de estados:

- **FIND_WALL**: el robot avanza en línea recta hasta detectar una pared al frente ($d_{\text{front}} < 0{,}45$ m). Este estado inicial es necesario para que el robot tenga una pared de referencia antes de comenzar el seguimiento.
- **TURN_RIGHT**: el robot gira sobre sí mismo hacia la derecha hasta que el frente queda libre. Para evitar oscilaciones, existe un contador mínimo de iteraciones (`MIN_TURN_TICKS = 15`) antes de poder salir del estado.
- **FORWARD**: el robot avanza siguiendo la pared izquierda mediante un controlador proporcional. El error se define como $e = d_{\text{left}} - d_{\text{desired}}$, de forma que si la pared está lejos el robot gira a la izquierda y si está cerca gira a la derecha.

Los parámetros principales del nodo son:

Parámetro	Valor	Descripción
`LINEAR_SPEED`	0.12 m/s	Velocidad lineal de avance
`ANGULAR_SPEED`	0.40 rad/s	Velocidad angular de giro
`WALL_DIST`	0.35 m	Distancia deseada a la pared izquierda
`FRONT_THRESH`	0.45 m	Umbral de obstáculo al frente
`KP`	0.8	Ganancia del controlador proporcional
`MIN_TURN_TICKS`	15	Iteraciones mínimas en estado de giro

Cuadro 1.2: Parámetros del nodo `res_maze.py`

Para lanzar el ejercicio:

```
# Terminal 1 - Simulacion
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_1.launch.py

```

```
# Terminal 2 - Solver
source /workspace/ros2_ws/install/setup.bash
ros2 run maze_pkg res_maze
```

[CAPTURA: Gazebo con el Turtlebot 3 Waffle en el laberinto 1 (estado inicial)]

Figura 1.2: Turtlebot 3 Waffle en el laberinto 1 — estado inicial

[CAPTURA: Gazebo con el Turtlebot 3 Waffle habiendo salido del laberinto 1]

Figura 1.3: Turtlebot 3 Waffle resolviendo el laberinto 1

El enlace al vídeo de la ejecución se puede encontrar en: ENLACE_VIDEO_MAZE1_WAFFLE

1.3 Ejercicio 2: Laberinto 2 y análisis de limitaciones

Se ejecuta el mismo nodo `res_maze.py` con el segundo mapa:

```
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_2.launch.py
```

[CAPTURA: Gazebo con el Turtlebot 3 Waffle en el laberinto 2]

Figura 1.4: Turtlebot 3 Waffle en el laberinto 2

Pregunta 14: Problemática observada en el laberinto 2

El laberinto 2 presenta una topología diferente al primero. La principal problemática observada es la presencia de **bucles cerrados** y pasillos simétricos que pueden llevar al robot a girar indefinidamente por la misma ruta sin encontrar la salida. El algoritmo

de seguimiento de pared izquierda es completo para laberintos simplemente conexos (sin islas interiores), pero falla cuando el laberinto contiene paredes desconectadas del perímetro, ya que el robot puede quedar atrapado orbitando alrededor de una isla interior.

Pregunta 15: ¿Es el robot capaz de resolver este laberinto?

El robot **no es capaz** de resolver el laberinto 2 con el algoritmo de seguimiento de pared izquierda. Como ya se ha dicho en la parte anterior se observa que la topología es distinta al primer ejercicio por lo cual la estrategia debería de ser diferente, se ha pensado en añadir a la máquina de estados un nuevo de estados en lo que implicaria un contador cada cierto número de giros a la izquierda seguidos en un periodo de tiempo corto cambiar de rumbo.

En caso de no poder resolverlo, la información adicional que necesitaría el robot para lograrlo sería:

- Un **mapa previo** del entorno (navegación basada en mapa, SLAM).
- **Localización global** para detectar si ha visitado una posición anteriormente y así evitar repetir rutas (algoritmo de Pledge o similares).
- Capacidad de **marcar el camino recorrido** para implementar estrategias como *Trémaux's algorithm*.

El enlace al vídeo de la ejecución se puede encontrar en: `ENLACE_VIDEO_MAZE2_WAFFLE`

1.4 Ejercicio 3: Repetición con Turtlebot 3 Burger

Se repiten los ejercicios anteriores usando el modelo `Burger`:

```
export TURTLEBOT3_MODEL=burger
ros2 launch maze_pkg maze_1.launch.py
# Terminal 2:
ros2 run maze_pkg res_maze
```

[CAPTURA: Gazebo con Turtlebot 3 Burger en el laberinto 1]

Figura 1.5: Turtlebot 3 Burger en el laberinto 1

Pregunta 16: Diferencias observadas entre Waffle y Burger en el laberinto

Las diferencias observadas entre ambos modelos al resolver el laberinto son las siguientes:

- **Dimensiones físicas**: el Burger es más pequeño y estrecho que el Waffle, lo que le permite navegar con mayor holgura por pasillos estrechos y cometer menos colisiones laterales.
- **LiDAR**: ambos modelos montan el mismo sensor LiDAR con 360° de cobertura, por lo que la información de percepción es equivalente.
- **Dinámica de giro**: el Burger, al tener menor separación entre ruedas (*wheel separation*), realiza giros más cerrados. Esto puede ser ventajoso en espacios reducidos pero también más sensible a errores en la odometría.
- **Velocidad y estabilidad**: el Waffle, al ser más pesado y ancho, es más estable a altas velocidades. El Burger tiende a oscilar más con los mismos parámetros del controlador.
- **Comportamiento en el laberinto**: [Completar tras la ejecución — indicar si el Burger resuelve el laberinto con mayor o menor facilidad, si choca con paredes, si los parámetros necesitan ajuste, etc.]

El enlace al vídeo de la ejecución con Burger se puede encontrar en: `ENLACE_VIDEO_BURGER`

1.5 Ejercicio 4: Laberinto propio

Se ha generado un laberinto personalizado usando el generador online mazegenerator.net y se ha implementado como modelo SDF en Gazebo.

[CAPTURA: imagen del laberinto generado en mazegenerator.net]

Figura 1.6: Laberinto personalizado generado con mazegenerator.net

Para lanzar el laberinto personalizado:

```
# Con Waffle
export TURTLEBOT3_MODEL=waffle
ros2 launch maze_pkg maze_3.launch.py

# Con Burger
export TURTLEBOT3_MODEL=burger
ros2 launch maze_pkg maze_3.launch.py
```

[CAPTURA: Gazebo con el laberinto personalizado y el Waffle]

Figura 1.7: Turtlebot 3 Waffle resolviendo el laberinto personalizado

[CAPTURA: Gazebo con el laberinto personalizado y el Burger]

Figura 1.8: Turtlebot 3 Burger resolviendo el laberinto personalizado

El enlace al vídeo de la ejecución con el laberinto personalizado: `ENLACE_VIDEO_MAZE3`